

Odczytywanie tablicy rejestracyjnej ze zdjęcia

Deep learning w Chmurze

Joanna Dagil Maja Szerszeń

17 czerwca 2026

Spis treści

- 1 Streszczenie
- 2 Wstęp / wprowadzenie
- 3 Opis kontekstu i celu projektu
- 4 Literatura
- 5 Opis danych
- 6 Opis metod
- 7 Opis wdrożenia metod / modelu
- 8 Wyniki
- 9 Wnioski

Celem projektu jest automatyczne **odczytanie numeru tablicy rejestracyjnej** z fotografii pojazdu.

Zastosowano **podejście dwustopniowe**: najpierw detekcja tablicy na zdjęciu, następnie rozpoznanie pojedynczych znaków.

Oba etapy oparto na modelu YOLO11 (Ultralytics); całość trenowana i uruchamiana w **środowisku chmurowym** (Kaggle / Colab, GPU Tesla T4).

Kluczowy pomysł: detektor znaków wzmocniono **danymi syntetycznymi** (generator tablic z automatycznymi bounding boxami) oraz **pseudo-etykietowaniem** prawdziwych cropów.

Wyniki: detekcja tablicy **mAP@0.5 = 0,97**, odczyt całej tablicy **exact match $\approx$ 61%**, błąd na poziomie znaku **CER $\approx$ 0,15**.

Automatyczne rozpoznawanie tablic rejestracyjnych (ANPR) jest powszechnie stosowane w systemach parkingowych, kontroli dostępu, pobierania opłat drogowych i monitoringu ruchu.

Zadanie jest trudne, ponieważ na zdjęciach występują:

- różne kąty, perspektywa i krzywizna tablicy,
- zmienne warunki oświetleniowe oraz niska rozdzielczość,
- rozmycie, szum i artefakty kompresji,
- podobne do siebie znaki (np. 0/0, Q/0, X, 8/B).

W projekcie sprawdzamy, czy lekkie modele detekcji jednofazowej w połączeniu z danymi syntetycznymi pozwalają zbudować pipeline odczytu tablic.

Zbudowanie systemu, który z fotografii pojazdu zwraca ciąg znaków odpowiadający tablicy rejestracyjnej.

Cele szczegółowe:

- wytrenowanie detektora położenia tablicy (YOLO11n),
- wytrenowanie detektora 36 klas znaków (cyfry 0–9, litery A–Z),
- rozwiązanie problemu **małej liczby oznaczonych znaków** poprzez dane syntetyczne i pseudo-etykietowanie,
- ilościowa ocena detekcji oraz tekstowa ocena odczytu (exact match, CER).

- Khanam, R., Hussain, M. (2024). *YOLOv11: An Overview of the Key Architectural Enhancements*. arXiv:2410.17725.
<https://arxiv.org/abs/2410.17725>
- Jocher, G., Qiu, J. (2024). *Ultralytics YOLO11* (Version 11.0.0) [Computer software]. GitHub.
<https://github.com/ultralytics/ultralytics>
- Arbab, B. A. *License Plate Detection Dataset (10,125 Images)*. Kaggle.
<https://www.kaggle.com/datasets/barkataliarbab/license-plate-detection-dataset-10125-images>
- Yazdani, N. *License Plate Text Recognition Dataset*. Kaggle.
<https://www.kaggle.com/datasets/nickyazdani/license-plate-text-recognition-dataset>

Pierwszy zbiór danych – *License Plate Detection Dataset (10,125 Images)* zawiera adnotacje lokalizacji tablic na zdjęciu, które pozwalają na **detekcję** tablicy na obrazie. Nie zawiera jednak oznaczenia faktycznych numerów tablic.

Drugi zbiór danych – *License Plate Text Recognition Dataset* (N. Yazdani) – zawiera tysiące prawdziwych cropów tablic z etykietami **tekstowymi** (bez bounding boxów znaków). Wykorzystany do pseudo-etykietowania (sekcja *Opis metod*).

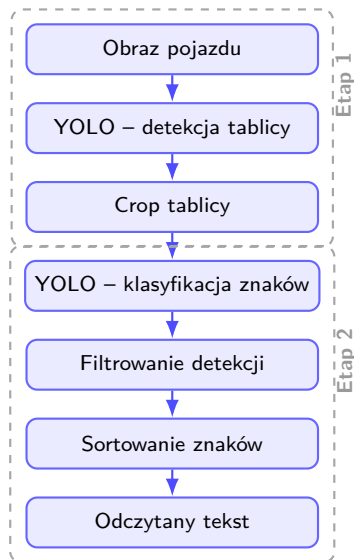
W celu zmierzenia **jakości odczytywania numerów** posłużyliśmy się analizą na **próbce ręcznie oznaczonych tablic** (*manual ground truth*).

Dokładniejsze opisy danych przy opisie metody z nich korzystającej.

Opis metod

W projekcie zastosowano podejście dwustopniowe, ponieważ problem odczytywania tablicy rejestracyjnej składa się z dwóch zadań:

- **detekcji tablicy** – znalezienia miejsca, w którym tablica znajduje się na obrazie,
- **rozpoznania tekstu** – odczytania znaków z wyciętego fragmentu tablicy.



Etap 1 – detekcja tablicy (YOLO11n)

Do wykrywania położenia tablicy rejestracyjnej wykorzystano model YOLO11n z biblioteki Ultralytics.

- YOLO jest jednofazowym (one-stage) modelem detekcji obiektów.
- Model uczony był detekcji jednej klasy: `License_Plate`.
- Zwraca ramki ograniczające (*bounding box*) oraz poziom pewności detekcji (*confidence score*).
- Wybrano wariant YOLO11n (nano, $\sim 2,6$ mln parametrów), ponieważ jest lekki i szybki, co ułatwia eksperymenty w chmurze.

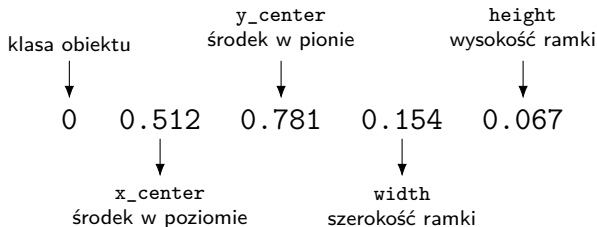
Konfiguracja treningu: 30 epok, `imgsz = 640`, `batch = 16`, wagi startowe `yolo11n.pt`, `patience = 10`.

License Plate Detection Dataset (10,125 Images)

Zdjęcia pojazdów z oznaczonym położeniem tablic rejestracyjnych.

Dane są zapisane w formacie zgodnym z YOLO. Dla każdego obrazu istnieje odpowiadający mu plik tekstowy z etykietą.

Podział: **train** / **valid** (2048 obrazów) / **test** (1020 obrazów), jedna klasa: `License_Plate`.



Wartości `x_center`, `y_center`, `width` i `height` są znormalizowane do rozmiaru obrazu, czyli mają wartości od 0 do 1.

Crop tablicy z marginesem

Po wykryciu tablicy przez model YOLO wycinany jest fragment obrazu odpowiadający przewidzianej ramce ograniczającej.

- Crop ogranicza dalsze przetwarzanie tylko do obszaru tablicy.
- Surowe cropy YOLO są bardzo ciasne – znaki dotykają krawędzi, co pogarsza rozpoznawanie. Dlatego wycinamy tablice ponownie z **marginesem $\approx 12\%$** wokół boxa.
- Każdy crop jest dodatkowo **wyrównywany pod kątem** obliczonym z linii znaków (deskew na etapie odczytu).
- Odrzucono wcześniejszą rektyfikację konturową (perspektywiczną) – psuła dobre cropy i nie prostowała krzywych tablic.

Etap 2 – detekcja znaków (Char-YOLO)

Drugi model wykrywa i klasyfikuje pojedyncze znaki na wyciętej tablicy.

- Wariant YOLO11s (small, $\sim 9,4$ mln parametrów) – znaki są małe, większa sieć pomaga.
- 36 klas: cyfry 0–9 oraz litery A–Z.
- Większa rozdzielczość wejścia ($\text{imgsz} = 480$) zwiększa znaki na mapach cech.
- Po detekcji następuje **filtrowanie** (NMS + reguły geometryczne) i **sortowanie** znaków w naturalnej kolejności czytania.
- Obsługa tablic **dwurzędowych**: górny wiersz $\rightarrow$ dolny wiersz.

Problem: dataset prawdziwych znaków jest mały (ok. 146 obrazów), więc 36-klasowa detekcja uczona tylko na nich działa słabo. Rozwiązanie na kolejnych slajdach.

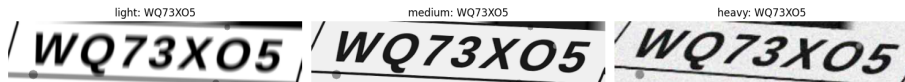
zawiera tysiące prawdziwych cropów tablic z etykietami **tekstowymi** (bez bounding boxów znaków). Wykorzystany do pseudo-etykietowania.

Etykiety znaków: klasy 0–35 odpowiadające cyfrom 0–9 oraz literom A–Z (CHAR_CLASSES).

Generator syntetycznych tablic

Aby zwiększyć liczbę danych treningowych **bez ręcznego oznaczania**, zbudowano generator syntetycznych tablic.

- Tablica renderowana jest **znak po znaku**, więc generator zna dokładne współrzędne każdego znaku → bounding boxy powstają automatycznie.
- Kolejność operacji:
 - 1 render czystej tablicy + boxy znaków,
 - 2 transformacje **geometryczne** (perspektywa, rotacja) – boxy przekształcane tą samą macierzą,
 - 3 degradacje **fotometryczne** (blur, szum, niska rozdzielczość, JPEG) – nie zmieniające geometrii.



Wizualizacja poziomów degradacji wygenerowanych tablic rejestracyjnych

Generator syntetycznych tablic

Wygenerowano **12 000** obrazów (.jpg + etykiety YOLO), w tym **~30% tablic dwurzędowych**.



Porównanie syntetyków z prawdziwymi cropami

Syntetyki trafiają **tylko do zbioru treningowego** – val/test pozostają w 100% prawdziwe.

Drugie źródło prawdziwych danych treningowych – **za darmo**.

- Drugi dataset ma tysiące prawdziwych cropów z etykietami **tekstowymi**, ale bez boxów znaków.
- **Trik:** przepuszczamy te cropy przez świeżo wytrenowany Char-YOLO. Jeśli odczytany ciąg **dokładnie zgadza się** z etykietą tekstową, to detekcje były poprawne → przewidziane boxy stają się nowymi etykietami treningowymi.
- Zgodność tekstu działa jak **automatyczny filtr jakości** (zła detekcja = zły tekst = odrzut). Próg pewności: $\text{conf} = 0,50$.
- Następnie dotrenowanie modelu o pseudo-etykiety (syntetyki + 146 prawdziwych + pseudo-etykietowane prawdziwe).

Środowisko chmurowe. Notatnik wykrywa automatycznie środowisko (Kaggle / Colab / lokalne) i ustawia ścieżki danych oraz zapisu wyników. Trening prowadzono na GPU **Tesla T4** (Ultralytics 8.4, PyTorch + CUDA).

Organizacja pipeline'u:

- Etap 1 (tablica) i Etap 2 (znaki) trenowane niezależnie; wagi zapisywane jako `plate_detector_best.pt` oraz `char_yolo_best.pt`.
- Możliwość pominięcia treningu przez wczytanie gotowych wag z wejścia (oszczędność czasu GPU).
- Oba modele pakowane do jednego ZIP-a do ponownego użycia / obrony.

Postprocessing odczytu: zamiana na wielkie litery, usunięcie spacji i znaków specjalnych, pozostawienie wyłącznie [A-Z0-9].

"PCZ DSq8" → "PCZDSQ8"

Wyniki – detekcja tablicy (YOLO11n)

Metryki detekcji tablicy (jedna klasa License_Plate):

Zbiór	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Walidacyjny	0,983	0,948	0,970	0,686
Testowy	0,990	0,949	0,972	0,692

- Bardzo wysoka skuteczność wykrywania samej tablicy ($\text{mAP@0.5} \approx 0,97$).
- Niższe mAP@0.5:0.95 wynika z surowszego progu IoU – lokalizacja jest nieco mniej precyzyjna, ale w zupełności wystarcza do cropowania.

Wyniki – detekcja znaków (Char-YOLO)

Metryki detekcji 36 klas znaków (val/test w 100% prawdziwe):

Zbiór	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Walidacyjny	0,818	0,792	0,838	0,605
Testowy	0,933	0,889	0,959	0,680

- Dane syntetyczne + pseudo-etykietowanie pozwoliły wytrenować 36-klasowy detektor mimo małej liczby prawdziwych oznaczeń.
- Najtrudniejsze są znaki łatwe do pomylenia: 0/0, Q, X, 8/B.

Wyniki – odczyt całej tablicy (tekst)

Ewaluacja **tekstowa** na ręcznie oznaczonej próbce (**177 przykładów**):

Metryka	Wartość
Exact match (cała tablica poprawna)	0,610
Similarity (podobieństwo ciągów)	0,877
CER (Character Error Rate)	0,147

- Ok. **61%** tablic odczytanych **w pełni poprawnie**.
- Średnio **~15% znaków** z błędem (CER) – najczęściej pojedyncza pomyłka na tablicę.
- Wysokie *similarity* (0,88) oznacza, że nawet błędne odczyty są zwykle bliskie prawdziwemu numerowi.

$CER = \text{odległość edycyjna} / \text{długość ground truth}$ (im niżej, tym lepiej).

- Dwustopniowy pipeline oparty na YOLO11 **skutecznie odczytuje tablice** – detekcja tablicy niemal bezbłędna ($mAP@0.5 \approx 0,97$).
- **Dane syntetyczne** okazały się kluczowe: pozwoliły obejść problem małego zbioru oznaczonych znaków bez ręcznego etykietowania.
- **Pseudo-etykietowanie** dostarczyło dodatkowych, darmowych prawdziwych danych, a zgodność tekstu zadziałała jak automatyczny filtr jakości.
- Wąskim gardłem pozostaje **etap odczytu znaków** (61% exact match): pojedyncze pomyłki na trudnych/krzywych tablicach obniżają dokładność całej tablicy.

Kierunki dalszych prac:

- więcej i bardziej realistycznych syntetyków (czcionki, tła, degradacje),
- lepsza rektyfikacja silnie przekrzywionych tablic,
- walidacja formatu numeru (reguły rejestracji) jako postprocessing.

Dziękujemy za uwagę